

M10 / LEARNER BOOK

Docker and Apache Airflow

Standalone Zero-to-Job-Ready Data Engineer 2026

19 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE10-01 Why containers exist	4
DE10-02 Images and containers	7
DE10-03 Writing a Dockerfile	10
DE10-04 Ports	13
DE10-05 Volumes and persistence	16
DE10-06 Networks and service communication	19
DE10-07 Compose, configuration and secrets	22
DE10-08 Why orchestration exists	25
DE10-09 Airflow architecture and UI	28
DE10-10 DAGs, tasks and dependencies	31
DE10-11 TaskFlow, operators and task communication	34
DE10-12 Scheduling and data intervals	37
DE10-13 Retries, timeouts and failure behavior	40
DE10-14 Parameters and task communication	43
DE10-15 Connections, variables and secrets	46
DE10-16 Sensors and waiting	49
DE10-17 Logs and debugging	52
DE10-18 Catchup, backfills and reprocessing	55
DE10-19 Integration and production-style orchestration	58

START HERE

How to use this module

Open the matching practice folder from the course root: 08_M10/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Outcome Package a small worker with Docker concepts, explain image/container/port/volume/network/Compose boundaries, author and debug an Airflow 3.3.1 TaskFlow DAG, derive partition paths from data intervals, use bounded failure behavior and defend safe backfill/reprocessing.

This review release certifies the pure-Python simulator and static/compile controls, but not live Docker or Airflow runtime behavior. Collect real networking, volume, Compose, scheduler/UI/DagRun, sensor and backfill evidence on the learner PC under WSL2.

Practice: 08_M10/practice/ Quick revision:

12_REVISION_AND_TESTS/M10/M10_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M10/PROTECTED_OPEN_AFTER_ATTEMPT/M10_Module_Revision_Test_Answers.pdf

- Watch the mapped real visual only when it helps you picture the interface or workflow. The learner book remains sufficient if the video is unavailable.
- Type/run the supplied local simulator and static QA. On your PC, add the real Docker/Airflow runtime evidence when the lesson calls for it.
- Close video/review/AI solving before the independent task. Save the first genuine attempt, including failures.
- Validate counts, partition identity, side effects, retry behavior and logs. A green command alone is not proof of data correctness.
- Use Quick Revision later. Take the Module Revision Test closed-book. G04 is a separate protected boundary assessment after M10.

Airflow content is aligned to Apache Airflow 3.3.1 and the stable airflow.sdk DAG-authoring interface. Video-first rule: useful real videos stay available even when a bounded chapter range is not verified. The three module visuals below are optional support; current official documentation remains the syntax/runtime authority.

Airflow content is aligned to Apache Airflow 3.3.1 and the stable airflow.sdk DAG-authoring interface. Lesson-level version-sensitive behavior is taught from current official documentation. The two long recordings below are optional module visuals only; because reliable chapter boundaries were not verified, they are not assigned as vague per-lesson watch ranges.

Current-generation Airflow visual support. Optional enrichment only; no invented chapter timestamp. Official Airflow 3.3.1 docs

DE10-01

Why containers exist

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A Python script that works on one laptop can still fail on another because the second machine has a different Python version, missing system libraries, different environment variables or an unexpected filesystem layout. A container gives the program a declared runtime boundary. It does not magically make bad code reliable, but it reduces the number of hidden machine assumptions you need to remember.

Picture it in your head

Picture shipping a small workshop in a sealed crate. The crate contains the tools and instructions needed by the worker. The host machine still supplies the ground, electricity and kernel, but the workshop no longer depends on whatever random tools happen to be lying around the host.

Words before we use them

image = reusable filesystem/runtime package; container = one running instance of an image; registry = place that stores/distributes images; host = machine running the container engine; isolation = separate process/filesystem/network views, not a separate physical computer.

Follow with me once

First compare two runbooks. Runbook A says 'install Python, pip install these things, set these variables, then hope the path is right.' Runbook B says 'build this Dockerfile, then run this image with this configuration.' In the supplied Docker lab, the worker has only one job: fetch a CSV from the source service and write a report. The image packages the worker code; the URL and output destination stay outside the image as configuration.

```
docker version
docker run --rm hello-world
docker image ls
docker ps -a
```

What you should see

On a learner PC with Docker working, hello-world proves the engine can pull an image and start a container. `docker image ls` shows stored images; `docker ps -a` shows container instances. These commands require a working Docker engine on the learner PC. Save their real output there; static course checks do not substitute for runtime evidence.

Break it / common failure

A common mistake is saying 'the container includes its own operating system.' Linux containers usually share the host kernel. Another mistake is treating a container as durable storage. If business data exists only in a disposable writable layer, removing the container can remove that state.

Reason through a realistic incident

A new teammate clones the repository on a clean laptop. The Python script fails outside Docker because their interpreter and packages differ from yours, but the image build succeeds. That is useful evidence that the container removed host-runtime assumptions. Now imagine the image still produces a wrong total: Docker cannot repair transformation logic. Separate environment reproducibility from business correctness before celebrating the fix.

Why the repair is safe

Containers improve reproducibility only when the build is reproducible, the configuration boundary is explicit, and persistent state is handled separately. A containerized pipeline can still be non-idempotent, insecure or logically wrong.

Try it yourself

Write a before/after environment story for `worker.py`: list what must be installed/configured without Docker, then list what moves into the image and what deliberately remains runtime configuration. Explain one problem containers solve and one problem they do not solve.

Fresh changed retry

A colleague says, 'We use Docker, so dependency pinning no longer matters.' Refute the statement using the difference between an image recipe and an unpinned package install inside that recipe.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Containers improve reproducibility only when the build is reproducible, the configuration boundary is explicit, and persistent state is handled separately. A containerized pipeline can still be non-idempotent, insecure or logically wrong.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-02

Images and containers

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

An image and a container are related but not interchangeable. Confusing them causes learners to delete the wrong thing, rebuild when they only needed to restart, or assume runtime changes altered the original image.

Picture it in your head

An image is a cookie cutter plus recipe; a container is one cookie made from it. You can make many containers from one image. Changing one running container does not silently rewrite the image used to create the next one.

Words before we use them

layer = filesystem change in an image build; tag = human-readable image reference; container writable layer = ephemeral changes made by one container; immutable artifact = build result treated as fixed rather than edited in place.

Follow with me once

The learner lab builds one `m10-worker:v1` image and can create many short-lived worker containers from it. The input URL changes through an environment variable. That means the same image can process a different source without rebuilding. Inspecting image history and container configuration teaches you which decision belongs at build time versus run time.

```
docker build -t m10-worker:v1 .
docker image inspect m10-worker:v1
```

```
docker run --rm -e SOURCE_URL=http://example.invalid/orders.csv m10-worker:v1
```

What you should see

The image remains after a short-lived container exits. A failed run caused by a deliberately invalid URL does not imply the image vanished. Re-running with corrected runtime configuration creates a new container from the same image.

Break it / common failure

Do not debug every container problem by rebuilding. A wrong environment variable is a runtime-config problem. Conversely, editing a host source file will not change an already-built image unless that file is bind-mounted or the image is rebuilt.

Reason through a realistic incident

An engineer shells into a running container, installs a package manually and says the image has been fixed. Stop and test the claim: create a second container from the same image. The package is absent because the manual change lived only in the first container writable layer. A durable repair belongs in the Dockerfile/build inputs, followed by a new image tag and a fresh container.

Why the repair is safe

Promotion usually moves immutable image candidates between environments while environment-specific configuration changes outside the image. This makes rollback and audit much easier than hand-editing running machines.

Try it yourself

Draw an image-to-container one-to-many diagram. For five changes - Python base version, application source, input URL, host port and API token - classify whether each normally belongs to image build, runtime configuration or secret management.

Fresh changed retry

Imagine two containers created from the same image produce different output because they were given different `SOURCE_URL` values. Explain why this does not violate image immutability.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry.

Preserve the failed attempt beside the repair.

Ready-made revision

Promotion usually moves immutable image candidates between environments while environment-specific configuration changes outside the image. This makes rollback and audit much easier than hand-editing running machines.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-03

Writing a Dockerfile

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A Dockerfile is executable infrastructure documentation: it turns source files and build instructions into an image. The order of instructions affects cache reuse, attack surface and whether another engineer can reproduce the same runtime.

Picture it in your head

Picture an assembly line. Each instruction adds a controlled step to the product. If you keep moving the first workstation, every later station must repeat its work; if stable steps come first, later rebuilds can reuse them.

Words before we use them

base image = starting filesystem/runtime; WORKDIR = default working directory; COPY = add build-context files; RUN = command executed while building; USER = runtime user; CMD = default process; build context = files Docker may send to the builder.

Follow with me once

Open `practice/docker/Dockerfile`. It starts from `python:3.13-slim`, creates an unprivileged user, copies only `worker.py`, and runs the worker with `exec-form CMD`. The worker uses only the Python standard library, so the example avoids fake dependency complexity and lets you focus on the build boundary.

```
docker build -t m10-worker:v1 practice/docker
docker history m10-worker:v1
```

```
docker inspect m10-worker:v1 --format '{{.Config.User}}'
```

What you should see

The image should build from the documented context and report appuser as the runtime user. `docker history` should show the build steps. The build is not considered proven in this artifact environment because Docker is absent here; the Dockerfile is statically checked instead.

Break it / common failure

Do not COPY . . . blindly from a large project that contains .env, virtual environments or evidence files. Do not put passwords in ENV or RUN instructions; image history and layers can preserve them. Do not use root merely because it avoids permissions learning.

Reason through a realistic incident

A Dockerfile works but rebuilds slowly and accidentally includes a local .env. Inspect build context and instruction order. The correct repair is not 'use more CPU'; exclude unnecessary/sensitive files, copy stable dependency metadata before frequently changing source, and rebuild. Then inspect image history/config rather than assuming the secret disappeared because the final app starts normally.

Why the repair is safe

A professional Dockerfile makes the runtime boundary reviewable: known base, minimal context, deterministic dependency install, explicit work directory, non-root user and a clear foreground process.

Try it yourself

Explain every instruction in the supplied Dockerfile in one sentence. Then modify a working copy so worker.py is copied to /opt/m10/worker.py; update WORKDIR/CMD consistently and predict what breaks if only one path is changed.

Fresh changed retry

Add a requirements file containing one package. Reorder the Dockerfile so dependency metadata is copied/installed before frequently changing application source. Explain the cache benefit without claiming the cache proves packages are pinned.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

A professional Dockerfile makes the runtime boundary reviewable: known base, minimal context, deterministic dependency install, explicit work directory, non-root user and a clear foreground process.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-04

Ports

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A service listening inside a container is not automatically reachable from the host. Port publishing is an explicit network boundary. The most common beginner error is mixing up the host port with the port where the process listens inside the container.

Picture it in your head

Picture an apartment building. The container process listens at an apartment door (container port). Port publishing creates a reception-desk rule that forwards one building entrance number (host port) to that apartment door.

Words before we use them

container port = where the process listens inside container namespace; host/published port = host-side entry point; EXPOSE = image metadata/documentation, not automatic publication; localhost = the current network namespace, not a universal name for 'my laptop'.

Follow with me once

The supplied Compose lab publishes the source service as `_${SOURCE_PORT:-8090}:8000`. From the host you can request `http://localhost:8090/orders.csv`. From the worker container you must use `http://source:8000/orders.csv`. The two addresses reach the same service from different network locations.

```
docker compose up -d source
docker compose ps
```

```
# host: http://localhost:8090/orders.csv
# worker network: http://source:8000/orders.csv
```

What you should see

`docker compose ps` should show a host mapping similar to `0.0.0.0:8090->8000/tcp`. Changing `SOURCE_PORT` changes only the host entry point; the service-to-service port remains 8000.

Break it / common failure

`EXPOSE 8000` does not publish port 8000. Publishing `8090:8000` also does not make the source process listen on 8090 inside the container. If a process listens only on `127.0.0.1` inside some container scenarios, publication may still not behave as you expect; always inspect the actual bind/listen configuration.

Reason through a realistic incident

The source service is healthy inside its container, but `http://localhost:8000` on the host fails while ``docker compose ps`` shows `8090->8000``. The evidence says the host must call port 8090. If the worker calls the source from the Compose network, it uses `source:8000`. One service therefore has two correct addresses depending on the caller's network namespace.

Why the repair is safe

Expose only interfaces that need host/external access. Databases and internal services often communicate on private container networks without publishing every port to the host.

Try it yourself

Change the host port to 8095 without changing the container port. State the host URL and the worker's internal URL. Explain why the worker should not start calling `localhost:8095`.

Fresh changed retry

Port 8090 is already occupied on your laptop. Design the smallest safe correction and explain why rebuilding the image would be unrelated to the conflict.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Expose only interfaces that need host/external access. Databases and internal services often communicate on private container networks without publishing every port to the host.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by

the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-05

Volumes and persistence

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Containers are replaceable processes. Business state should not depend on one container's writable layer surviving forever. Volumes and bind mounts deliberately separate persistent or shared data from the container lifecycle.

Picture it in your head

Imagine replacing a rented office every day. If the ledger is taped to the office wall, replacement loses it. A named storage locker survives office replacement. A bind mount is more like giving the office direct access to a specific host folder.

Words before we use them

named volume = Docker-managed persistent storage; bind mount = host path mounted into a container; writable layer = container-local ephemeral filesystem changes; read-only mount = mounted content the container may read but not modify.

Follow with me once

In `compose.yaml`, the source CSV folder is bind-mounted read-only because it is learner-controlled input. The worker writes its report into the named `report_data` volume. `docker compose down` removes containers but keeps the named volume; `docker compose down -v` intentionally deletes it.

```
docker compose up --build
docker volume ls
```

```
docker compose down
docker compose up --build
# deliberate reset only:
docker compose down -v
```

What you should see

After ordinary down and up, the named volume still exists. After down -v, it is removed. The source bind mount remains on the host because it was never Docker-managed volume data.

Break it / common failure

Never teach `docker compose down -v` as routine cleanup when the volume contains data you care about. Also avoid bind-mounting broad host folders with write access just to bypass permissions. Mount only what the service needs.

Reason through a realistic incident

A learner runs `docker compose down -v` to 'clean up' and loses a database volume. Recreate the timeline: down removes service containers/network but normally leaves named volumes; down -v explicitly removes project volumes. The recovery lesson is operational, not syntactic: destructive cleanup flags belong in documented reset procedures, and important volume data still needs backups.

Why the repair is safe

State ownership is an architectural decision. Volumes survive container replacement but do not by themselves provide backup, replication, retention or corruption recovery.

Try it yourself

Classify five paths - immutable input CSV, generated report, application source during development, database data directory, secret file - as image content, bind mount, named volume or secret mechanism. Defend each choice.

Fresh changed retry

A teammate says 'the database is in a container, so backups are unnecessary because the volume persists.' Explain why persistence and backup/recovery are different controls.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

State ownership is an architectural decision. Volumes survive container replacement but do not by themselves provide backup, replication, retention or corruption recovery.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-06

Networks and service communication

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Multi-container systems fail surprisingly often because one service calls `localhost` when it really needs another service. Compose gives services a shared default network and DNS-based service discovery so code can use stable service names instead of container IP addresses.

Picture it in your head

Inside each container, `localhost` means 'this room'. Calling another service is like calling another room by its directory name. Compose supplies the directory; hard-coded IP addresses are like memorizing a hotel room number that can change after checkout.

Words before we use them

service name = Compose DNS name; default network = automatically created Compose network; bridge network = common local Docker network driver; DNS/service discovery = name-to-address resolution inside the network.

Follow with me once

The worker uses `SOURCE_URL=http://source:8000/orders.csv`. `source` is the Compose service name. No container IP appears in the code. Bring the source service down and the worker loses the dependency; bring it back and the same name resolves to the replacement container.

```
docker compose up -d source
```

```
docker compose run --rm worker
docker compose exec source hostname
docker network ls
```

What you should see

With both services on the same Compose network, the worker can resolve source. The host-facing URL still uses localhost and the published port. The internal URL uses service name plus container port.

Break it / common failure

Do not fix a service-to-service failure by copying the current container IP into source code. Do not assume `depends_on` means the dependency is ready to serve; start order and readiness are different concepts. A health check or application retry may still be necessary.

Reason through a realistic incident

The worker suddenly cannot reach source after someone changed the Compose service name to feed. A copied container IP works briefly, then fails after recreation. The durable repair is to update the service-name configuration and keep name-based discovery. This incident demonstrates why ephemeral addresses are runtime details and why configuration should refer to stable service identities.

Why the repair is safe

Stable service discovery plus health/readiness checks is safer than coding around ephemeral addresses. Network boundaries also help you avoid publishing internal-only services to the host.

Try it yourself

Explain three addresses: `localhost:8090` from the host, `source:8000` from worker, and `localhost:8000` from inside source. State which namespace each `localhost` refers to.

Fresh changed retry

Rename the Compose service from source to feed. List every configuration reference that should change and every place that should not need an IP update.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Stable service discovery plus health/readiness checks is safer than coding around ephemeral addresses. Network boundaries also help you avoid publishing internal-only services to the host.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-07

Compose, configuration and secrets

Watch first

[1. DataTalksClub Docker/PostgreSQL workshop](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Docker visual Video assignment

Video assignment

[Docker visual - DataTalksClub workshop - Open video](#) Optional visual - no verified bounded range.

[Optional module video exists in WATCH GUIDE.md, but no bounded lesson segment is assigned because a](#) reliable chapter boundary was not verified. Do not watch a long recording just to satisfy this lesson.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Compose turns a collection of container commands into a version-controlled application definition. But a reproducible stack still needs disciplined configuration and secret boundaries; putting every value into YAML is not the same as managing it well.

Picture it in your head

Think of Compose as a stage manager's cue sheet: which actors exist, which props/volumes they use, which doors/ports open, what environment cues they receive and which actor must be healthy before another starts.

Words before we use them

service = one application component; compose.yaml = multi-service definition; interpolation = substituting environment values into config; secret = sensitive value needing restricted handling; health check = command that reports service readiness/health.

Follow with me once

Read `compose.yaml` with `docker compose config` before running it. The file defines source and worker services, a published source port, a read-only input bind mount, a persistent report volume and a source health check. `.env.example` documents safe non-secret values. The project intentionally contains no real credentials.

```
cd practice/docker
cp .env.example .env
```

```
docker compose config
docker compose up --build
docker compose logs
docker compose down
```

What you should see

`docker compose config` should resolve variables and show service/network/volume structure before any container starts. The worker should eventually write one report based on the served CSV. A clean clone should need only documented environment values, not hidden machine state.

Break it / common failure

Never commit real passwords merely because they are in `.env`. `.env` is configuration storage, not automatically a secret vault. Do not use Compose Variables as a substitute for production secret backends. Also do not call down `-v` casually.

Reason through a realistic incident

A stack works only on the original developer laptop because they relied on an untracked `.env` and an already-existing named volume. A second learner gets a different port and empty output. Reproducibility means documenting required non-secret config, validating the resolved Compose file with ``docker compose config``, and distinguishing intentional persistent state from hidden stale state.

Why the repair is safe

Version-control safe structure and names; inject environment-owned values at runtime; store real secrets in a secrets mechanism appropriate to the deployment environment. Reproducibility means a clean environment can rebuild the same topology from documented inputs.

Try it yourself

From a blank page, reconstruct a two-service Compose design: source + worker, internal service-name URL, one host port, read-only input and named output volume. Then compare to the supplied file and record only differences that matter.

Fresh changed retry

Move the host port and output volume name through environment interpolation. Explain why changing those values should not require rebuilding the worker image.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Version-control safe structure and names; inject environment-owned values at runtime; store real secrets in a secrets mechanism appropriate to the deployment environment. Reproducibility

means a clean environment can rebuild the same topology from documented inputs.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

Before DE10-08: install and start Airflow once

Use the supplied practice/airflow/AIRFLOW_SETUP_WSL.md. The learner route is WSL2 -> isolated venv -> Airflow 3.3.1 with official constraints -> airflow standalone -> localhost:8080 -> copy the supplied DAG -> verify dags list/import errors -> run and inspect logs. Do not continue to Airflow lessons until airflow version, UI access and DAG discovery are real on your PC.

```
cd "$M10_COURSE_DIR"
bash practice/airflow/install_airflow_331.sh
source practice/airflow/activate_m10_airflow.sh
airflow version
airflow standalone
```

_V

Apache Airflow 3.3.1 Quick Start [Open mapped source](#)

Official installation/standalone/UI authority. On Windows use WSL2.

DE10-08

Why orchestration exists

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Video assignment

Video assignment

Airflow 3 visual | Airflow 101 visual

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson reference first.](#)

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A cron line can start a script, but production workflows need more than start time: dependency order, retries, run history, backfill semantics, parameterization, observability and safe recovery. Orchestration is the control plane that coordinates work; it should not hide the business transformation logic.

Picture it in your head

Picture an airport control tower. The tower does not fly the planes or load the luggage; it coordinates when each movement is allowed, watches state, handles delays and keeps a history of what happened.

Words before we use them

orchestration = coordinating tasks/runs/dependencies; workflow = connected units of work; run = one execution instance; control plane = system coordinating work; idempotent = repeatable without multiplying logical state.

Follow with me once

Run `practice/simulator/scripts/orchestration_simulator.py` before touching Airflow. It processes date-partitioned order files, writes one deterministic report per partition, records attempts and can rerun a partition safely. This separates the workflow concept from one orchestration vendor.

```
python practice/simulator/scripts/orchestration_simulator.py \  
  --input-dir practice/simulator/input --output-dir practice/simulator/output  
  
ic
```

Three partitions should produce row counts 4, 3 and 5. Re-running should overwrite the same partition result with the same business totals rather than append duplicate logical output. That behavior is what later makes backfill/retry discussion concrete.

Break it / common failure

Do not move all transformation code into a huge DAG file just because Airflow can run Python. Do not make orchestration responsible for fixing bad source semantics. Keep the business work testable outside the scheduler where possible.

Reason through a realistic incident

A nightly script is launched by cron and sometimes starts before its source file exists. The team adds `sleep(900)` and now every run waits even when data arrives early. An orchestrator lets you model the dependency/wait condition explicitly, record run state and retry only the affected unit. The point is not that Airflow makes the transform correct; it makes execution state and dependencies observable.

Why the repair is safe

Orchestration adds controlled execution semantics around testable units. A good DAG is a readable dependency model, not a dumping ground for every line of business code.

Try it yourself

Draw extract -> validate -> publish for one date. Mark which step owns data correctness and which orchestration concerns are added around it: dependency, retries, run metadata and historical rerun.

Fresh changed retry

The pipeline now has two independent downstream reports after validation. Redraw the graph and explain why the orchestrator can run those two branches concurrently without changing the validation contract.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Orchestration adds controlled execution semantics around testable units. A good DAG is a readable dependency model, not a dumping ground for every line of business code.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-09

Airflow architecture and UI

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Airflow has multiple components because scheduling, state/API/UI and task execution are separate concerns. A learner does not need to memorize deployment diagrams, but must know where to look when a DAG is parsed, scheduled, run or shown as failed.

Picture it in your head

Think of a restaurant: the DAG file is the menu/work plan, the scheduler decides which tickets are ready, the metadata database records orders/state, execution infrastructure runs the work, and the UI/API lets operators inspect what happened.

Words before we use them

Dag = workflow definition; scheduler = creates/schedules task instances when dependencies/time allow; task instance = one task in one Dag run; metadata database = Airflow operational state; UI/API server = operational interface; executor/worker = mechanism that runs tasks.

Follow with me once

Open `practice/airflow/dags/m10_daily_orders.py`. It uses Airflow 3's stable `airflow.sdk` authoring namespace. On a learner PC with Airflow 3.3.1, the Dag should parse into a graph with extract, validate and publish-style work. The supplied DAG is compile/static-checked, but scheduler/UI execution remains a learner-PC proof. Do not mark the lesson runtime-complete until the DAG is discovered and run in Airflow.

```
# learner PC after Airflow setup
airflow version
airflow dags list
airflow dags list-import-errors
# inspect the UI Graph/Grid/Run details
```

What you should see

Airflow 3.3.1 should be reported on the pinned learner environment, the Dag should appear without import errors, and one test/manual run should expose task/run state in the UI. These are pending native-runtime proofs, not release claims made from this container.

Break it / common failure

When a Dag is missing, inspect import/parsing errors before reinstalling everything. When a task fails, distinguish Dag parse failure from task runtime failure. UI red does not tell you root cause; logs and task context do.

Reason through a realistic incident

A Dag file is in the expected folder but never appears in the UI. Before touching task logic, check Dag import errors. If import succeeds and the Dag appears but no run is scheduled, inspect schedule/start/catchup/pause state. If a task instance fails, inspect its log. Naming the lifecycle layer prevents a reinstall from masking a simple parse or schedule problem.

Why the repair is safe

Operational diagnosis improves when you name the layer first: definition/parse, scheduling, task execution, dependency/service, or data correctness. Airflow gives state; your engineering still has to interpret it.

Try it yourself

Write a troubleshooting decision tree for: Dag absent from UI, Dag present but no scheduled run, run exists but one task fails. Name the first evidence you would inspect in each case.

Fresh changed retry

The scheduler is healthy but a provider import inside one Dag is missing. Explain why restarting the whole machine is a weaker first response than checking Dag import errors and the environment where Airflow parses files.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Operational diagnosis improves when you name the layer first: definition/parse, scheduling, task execution, dependency/service, or data correctness. Airflow gives state; your engineering still has to interpret it.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-10

DAGs, tasks and dependencies

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

[Airflow 3 visual](#) | [Airflow 101 visual](#) [Video assignment](#)

[Video assignment](#)

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A DAG should encode real dependencies, not merely the order in which lines happened to be written. Directed and acyclic means work flows forward and cannot contain a dependency loop.

Picture it in your head

Imagine a construction plan: foundations must finish before walls; plumbing and electrical can proceed in parallel after walls; inspection waits for both. The graph captures prerequisites, not a screenplay.

Words before we use them

DAG = directed acyclic graph; task = schedulable unit of work; upstream/downstream = dependency direction; fan-out = one prerequisite feeding multiple tasks; fan-in = multiple prerequisites joining before a task.

Follow with me once

The supplied Dag uses small tasks rather than one monolith. Extract identifies the logical partition, validation checks expected controls, and publish records the result. The dependency is created by passing task results into downstream TaskFlow calls, making the graph visible from code.

```
partition = extract_partition()
validated = validate_partition(partition)
publish_partition(validated)
```

What you should see

The graph is linear for the base lab: extract -> validate -> publish. If validation fails, publish should not run. This is a dependency safety property, not merely a visual preference.

Break it / common failure

Do not create fake dependencies to force a pretty left-to-right diagram. Do not split work into tiny tasks that pass huge data through orchestration metadata. Task boundaries should reflect retries, ownership, resources, observability or meaningful dependency boundaries.

Reason through a realistic incident

A developer puts extract, validate and publish inside one giant task. Validation fails after an expensive extract, so retries repeat everything. Splitting at meaningful operational boundaries lets extract evidence be reused or independently diagnosed while publish remains blocked. Do not over-split either: a task boundary should buy retry, ownership, resource or observability value.

Why the repair is safe

Graph design is operational design: task boundaries determine what can retry independently, what can run concurrently, what evidence is isolated and how failures block downstream side effects.

Try it yourself

Redesign the graph so after validation it builds `finance_report` and `ops_report` in parallel, then one handoff waits for both. Draw it before writing syntax and state why each edge exists.

Fresh changed retry

A new alert task should run only after validation failure, not as an ordinary downstream success dependency. Describe the trigger/branching requirement rather than connecting it blindly to publish.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry.

Preserve the failed attempt beside the repair.

Ready-made revision

Graph design is operational design: task boundaries determine what can retry independently, what can run concurrently, what evidence is isolated and how failures block downstream side effects.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-11

TaskFlow, operators and task communication

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Airflow offers classic operators and the TaskFlow API. TaskFlow makes Python-centric DAGs easier to read by turning decorated functions into tasks and wiring returned values through XComs. The convenience does not mean XCom should become a bulk-data transport system.

Picture it in your head

Treat task return values like claim tickets, not freight trucks. A small path, count, ID or metadata object can coordinate tasks. A million-row dataframe belongs in durable storage, with a small reference passed between tasks.

Words before we use them

TaskFlow = decorated Python task authoring; operator = reusable task type; XCom = small task-to-task metadata exchange; XComArg = symbolic reference to a task result used while defining dependencies.

Follow with me once

`m10_daily_orders.py` imports `dag`, `task`, `Param` and `get_current_context` from `airflow.sdk`.
`extract_partition` returns a small dictionary containing partition/path facts.
`validate_partition` returns a small validation summary. No dataframe or raw CSV payload is returned through XCom.

```
from airflow.sdk import dag, task

@task
def extract_partition():
    return {'partition': '2026-09-01', 'path': '/data/orders_2026-09-01.csv'}
```

What you should see

Calling a decorated task while defining the Dag creates a task/XComArg relationship rather than immediately executing the function. In a real run Airflow stores task communication according to its XCom backend; our course keeps messages deliberately small.

Break it / common failure

Do not confuse Python function call appearance with ordinary eager execution at Dag-definition time. Do not return huge datasets just because it works in a toy local environment. Do not use a global Variable to pass per-run task output.

Reason through a realistic incident

A TaskFlow task returns a 500 MB pandas dataframe and local testing seems fine. Later the metadata/XCom path becomes slow and fragile. Move the dataset to durable storage and return only a URI/table name plus counts/checksum. The Dag still expresses the dependency, but Airflow coordinates references rather than becoming the bulk data transport layer.

Why the repair is safe

Use orchestration metadata for coordination; use data stores for data. This keeps scheduler/metadata systems small, makes retries clearer and avoids coupling workflow correctness to oversized XCom serialization.

Try it yourself

For five candidate values - partition date, 2 GB dataframe, output table name, validation row count, API password - decide XCom/small return, durable storage reference, Connection/secret, or something else. Defend each boundary.

Fresh changed retry

Replace a returned full list of 100,000 IDs with an object-storage/database path plus count/checksum metadata. Explain how this improves operational robustness.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Use orchestration metadata for coordination; use data stores for data. This keeps scheduler/metadata systems small, makes retries clearer and avoids coupling workflow correctness to oversized XCom serialization.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-12

Scheduling and data intervals

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

The most damaging scheduling bugs often come from using wall-clock 'today' inside a historical run. Airflow schedules work for a logical interval. A rerun of 2026-09-02 performed next week must still process the 2026-09-02 partition.

Picture it in your head

A Dag run is an envelope labeled with the business interval it represents. The worker may open the envelope later, but the label does not change to today's date merely because execution happens later.

Words before we use them

schedule = rule creating runs; data interval = time range one run represents; logical date = identifier associated with the run, not a command to use today's wall clock; catchup = creation of historical scheduled runs after a start boundary.

Follow with me once

The supplied Dag derives the partition from `data_interval_start` in task context. It never calls `datetime.now()` or `date.today()` to choose a business file. For a daily interval starting 2026-09-02 UTC, the task targets `orders_2026-09-02.csv` even if the task is actually executed days later.

```
ctx = get_current_context()
start = ctx['data_interval_start']
partition = start.date().isoformat()
path = f'/data/orders_{partition}.csv'
```

What you should see

Historical runs map to historical partitions deterministically. The Dag uses a timezone-aware start date. The course treats `data_interval_start/end` as the semantic boundary; it does not teach deprecated 'execution date means run-now time' intuition.

Break it / common failure

`date.today()` is wrong for replayable partition selection. An offset-naive start date creates ambiguity. Changing catchup without understanding what historical runs will be created can unexpectedly process or skip partitions.

Reason through a realistic incident

On September 10, an engineer reruns the failed September 2 Dag run. Code using `date.today()` reads September 10 input and overwrites September 10 output while the UI still labels the historical run as September 2. Deriving the path from `data_interval_start` prevents this silent historical corruption. This is why schedule semantics are a data-quality control.

Why the repair is safe

Interval-correct workflows make backfills, retries and audits possible. Time semantics are part of data correctness, not a scheduler trivia question.

Try it yourself

Given three runs representing Sep 1, Sep 2 and Sep 3 but physically executed on Sep 10, write the exact input path each must use. Then explain why wall-clock code would make all three collide.

Fresh changed retry

Change from daily to hourly processing. State what one data interval represents and how the partition path needs more than a date to stay unique.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Interval-correct workflows make backfills, retries and audits possible. Time semantics are part of data correctness, not a scheduler trivia question.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

Work through the three-interval schedule

The fixture dates are September 1, 2 and 3, 2026. With a daily data-interval timetable, the run for September 1 covers [September 1 00:00, September 2 00:00). The latest allowed interval start is September 3. Airflow end_date is an inclusive restriction on interval starts, so this teaching DAG uses September 3, not September 4. The September 3 interval becomes schedulable after it finishes on September 4. Rerunning that historical interval later still reads the September 3 fixture. The expected row counts are 4, 3 and 5. If a fourth run looks for September 4 input, inspect the timetable restriction; do not create invented input merely to make the scheduler green.

DE10-13

Retries, timeouts and failure behavior

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Retries are useful only for failures that may succeed later. Blind retries can duplicate side effects, hide deterministic defects and keep a bad workflow busy for hours. Timeouts bound work that hangs without reaching a clean failure.

Picture it in your head

Imagine calling a supplier. A busy signal may justify calling again after a pause. A permanently invalid account number should fail immediately. A call that never ends needs a time limit.

Words before we use them

transient failure = temporary condition; deterministic failure = same bad input/config repeatedly fails; retry delay = wait before next attempt; execution timeout = maximum task-run duration; idempotency = safe repeated execution.

Follow with me once

The local simulator can deliberately fail one partition once, then succeed on the bounded retry. The Airflow scaffold gives validation a limited retry count and timeout only where retry is safe. A data-contract failure should raise clearly and stop; it should not be retried until the source changes.

```
@task(retries=2, retry_delay=timedelta(seconds=30), execution_timeout=timedelta(minutes=5))
def extract_partition(...): ...
```

What you should see

A simulated transient error records one failed attempt and then success. A deterministic invalid-row condition fails validation and blocks publish. Re-running a successful partition does not multiply logical output.

Break it / common failure

Do not catch every exception and return success. Do not retry authentication errors or schema violations indefinitely. Do not add retries around a non-idempotent write without first defining duplicate/transaction behavior.

Reason through a realistic incident

A task receives a schema-invalid file and has `retries=5`. It fails identically six times, delaying downstream alerts. Another task gets a temporary connection reset and succeeds on retry. Same scheduler mechanism, different failure class. Design retries around recoverability and side-effect safety, and use timeouts for hangs rather than treating every failure as transient.

Why the repair is safe

Retry policy is part of side-effect design. Reliable orchestration distinguishes transient infrastructure failures from deterministic data/config defects and leaves enough evidence to diagnose both.

Try it yourself

Classify timeout, HTTP 503, missing required column, bad password, temporary network reset and negative quantity as retry/repair/fail categories. Explain where idempotency changes your decision.

Fresh changed retry

A publish task sends an external payment request and you cannot prove idempotency. Decide whether automatic retry is safe and name the external idempotency key/transaction control you would require.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Retry policy is part of side-effect design. Reliable orchestration distinguishes transient infrastructure failures from deterministic data/config defects and leaves enough evidence to diagnose both.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-14

Parameters and task communication

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A workflow needs run-time inputs, but parameters are not the same as task output. Params describe controlled run configuration; task returns/XCom describe facts produced during that particular run.

Picture it in your head

Before a trip, you choose destination and baggage limit - those are run parameters. During the trip, the airline produces gate number and actual bag count - those are run results. Mixing the two creates confusing ownership.

Words before we use them

Param = validated run configuration with defaults; run config = values supplied when triggering; XCom/task return = per-run task result; JSON Schema validation = rules constraining acceptable Param values.

Follow with me once

The Dag defines `min_rows` as an integer Param with a minimum. Validation reads the run-time value from context. The extract task returns partition/path metadata. This keeps operator intent separate from discovered run facts.

```
params={'min_rows': Param(1, type='integer', minimum=1)}
ctx = get_current_context()
min_rows = ctx['params']['min_rows']
```

What you should see

A manual trigger with an invalid Param should be rejected by Airflow's validation instead of reaching task code. A scheduled run uses the default. The value is small configuration, not a substitute for sending datasets between tasks.

Break it / common failure

Do not read manually supplied Param values from `dag.params` inside top-level definition code and assume you have run-time overrides; Airflow documentation distinguishes defaults from `DagRun/task` context. Do not store per-run task outputs in global Variables.

Reason through a realistic incident

A manual run needs `min_rows=100`, but code reads `dag.params` at parse time and keeps the default 1. The run-time override lives in task context, not in the static Dag object used during definition. This incident makes the ownership rule concrete: defaults help define the Dag; actual per-run Params are consumed when the task executes.

Why the repair is safe

Parameters make controlled variation explicit. Keep them small, typed and business-readable; keep observed run facts in task outputs/metadata and bulk data in data stores.

Try it yourself

Design Params for `min_rows`, `target_region` and `dry_run`. Give type/default/validation rules. Then list which facts should instead be task results: observed rows, output path, actual total.

Fresh changed retry

A teammate wants a Param containing 50,000 customer IDs. Redesign the interface so the Param contains a durable input reference plus validation metadata instead.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry.

Preserve the failed attempt beside the repair.

Ready-made revision

Parameters make controlled variation explicit. Keep them small, typed and business-readable; keep observed run facts in task outputs/metadata and bulk data in data stores.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by

the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-15

Connections, variables and secrets

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

External systems need addresses and credentials. Airflow Connections describe those systems; Variables hold global runtime configuration; secrets should come from protected mechanisms rather than being pasted into DAG code.

Picture it in your head

A Connection ID is a labeled key on the key rack; the actual password is stored behind controlled access. A Variable is a noticeboard setting. A task result is a note from one job to the next. They serve different ownership scopes.

Words before we use them

Connection = external-system connection metadata; Variable = global key/value runtime config; secrets backend = protected external store Airflow can query; connection ID = stable non-secret identifier used by DAG code.

Follow with me once

The learner Dag uses a connection ID such as `orders_store` in discussion/examples rather than embedding credentials. Safe code can commit the identifier; environment or Airflow admin config supplies the actual endpoint/password. The project includes `.env.example` only for non-secret demonstration values.

```
# Safe identifier in code
connection_id = 'orders_store'
# Do not commit:
# password = 'real-password-here'
```

What you should see

A clean repository contains no real password/token literals. Moving between environments should replace connection/secret configuration without editing business DAG logic. Global Variables are not used as a database for per-run task data.

Break it / common failure

If a secret was committed, moving it to a Variable after the fact does not erase Git history; rotate it and follow repository history-removal policy. Do not print connection URIs/tokens into task logs. Do not confuse environment interpolation with secret protection.

Reason through a realistic incident

A password is removed from the Dag and moved into an Airflow Connection, but old task logs and Git history still contain it. The engineering response includes credential rotation and history/log handling, not merely a code edit. Going forward the Dag uses a stable connection ID and retrieves protected values through the environment's Airflow connection/secrets configuration.

Why the repair is safe

Portable orchestration keeps code stable while environment-owned connection/secrets configuration changes behind stable identifiers. Least privilege and log sanitization remain required.

Try it yourself

Classify ten values: warehouse password, warehouse connection ID, alert threshold, partition date, output row count, Slack token, static table name, run-specific `dry_run`, source system URL, 2 GB result set. State where each belongs.

Fresh changed retry

Your company moves from Airflow DB-stored Connections to Vault. Explain why well-designed DAG code using the same connection ID may not need business-logic changes.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Portable orchestration keeps code stable while environment-owned connection/secrets configuration changes behind stable identifiers. Least privilege and log sanitization remain required.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-16

Sensors and waiting

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

[Airflow 3 visual](#) | [Airflow 101 visual](#) [Video assignment](#)

[Video assignment](#)

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

A workflow sometimes needs to wait for an external condition: a file arrival, upstream completion or data readiness. A sensor models waiting as a first-class task, but inefficient waiting can occupy resources for hours.

Picture it in your head

A sensor is a guard at a gate. The guard should check whether the delivery arrived, then step aside efficiently between checks rather than occupying the loading dock doing nothing.

Words before we use them

sensor = task that waits for condition; poke interval = how often condition is checked; timeout = maximum total wait; deferrable = waiting mode that frees a worker slot where supported; file sensor = waits for file/path condition.

Follow with me once

The supplied Airflow scaffold includes a separate `m10_wait_for_file.py` example using the standard provider `FileSensor` with a bounded timeout and deferrable mode. The main daily Dag can stay runnable without depending on that example. The point is to understand resource-safe waiting, not to add sensors everywhere.

```
FileSensor(
    task_id='wait_for_partition',
    filepath='/data/orders_2026-09-03.csv',
    poke_interval=60, timeout=1800, deferrable=True,
)
```

What you should see

A sensor succeeds only when its condition is true; it should fail after the declared timeout rather than wait forever. Deferrable support depends on the provider/runtime and must be proven in the learner environment.

Break it / common failure

Do not write `while True: sleep(...)` inside an ordinary worker task as your default waiting strategy. Do not use a 1-second poke interval for a file expected once per hour. Do not omit timeout and create endless invisible waiting.

Reason through a realistic incident

A file sensor checks every second for a partner file that normally arrives once per hour. Hundreds of unnecessary checks consume resources and clutter logs. Increase the poke interval, keep a meaningful timeout and use deferrable waiting when supported. Reliability includes the cost of waiting, not just whether the condition eventually becomes true.

Why the repair is safe

Waiting is capacity planning. A correct sensor defines condition, cadence, timeout, resource mode and failure ownership; otherwise a quiet wait can be as operationally expensive as a failed task.

Try it yourself

Choose sensible poke interval and timeout for three cases: file expected within 5 minutes, partner file within 2 hours, daily data warehouse table expected by 08:00. Defend resource vs detection-latency tradeoffs.

Fresh changed retry

The external platform can emit an event instead of being polled. Explain why event-driven/deferrable waiting may be better than frequent polling and what failure/timeout behavior still needs definition.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Waiting is capacity planning. A correct sensor defines condition, cadence, timeout, resource mode and failure ownership; otherwise a quiet wait can be as operationally expensive as a failed task.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-17

Logs and debugging

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Airflow 3 visual | Airflow 101 visual Video assignment

Video assignment

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Logs are evidence, not decoration. When a workflow fails, you need to reconstruct which run, task, partition, attempt and input caused the failure without leaking sensitive data into the log stream.

Picture it in your head

Think of a flight recorder: useful events have identity and sequence. 'Something failed' is not enough; a useful record says which run, what input boundary, what check failed and what recovery was attempted.

Words before we use them

structured context = consistent identifiers/fields in logs; attempt = retry number; traceback = exception call path; import error = Dag parse/load problem; task log = runtime evidence for one task instance.

Follow with me once

The simulator writes run summaries containing partition, row count, total, attempt and status. The Airflow task examples use Python logging and include partition/run context without printing secrets. For a missing Dag, check import errors; for a failed task, open that task instance log; for wrong data with green tasks, inspect validation controls rather than assuming orchestration success equals correctness.

```
logger.info('partition=%s rows=%s total=%s', partition, rows, total)
# never: logger.info('password=%s', password)
```

What you should see

A failure can be traced to a specific partition/attempt and compared with a later repaired attempt. The logs contain enough context to reproduce the problem but no password/token literals.

Break it / common failure

Do not log entire dataframes or response bodies containing PII/secrets merely because debugging is hard. Do not erase failed logs/evidence after repair. Do not read only the final exception line when the earlier stack/context shows the actual cause.

Reason through a realistic incident

A publish task fails after three retries. The final log line says only 'ValueError', but the earlier context line identifies partition 2026-09-03, row count 5 and one duplicate key. The fastest recovery comes from structured context plus the traceback. If logs had printed a real token during debugging, that token would now need rotation even though the task failure was unrelated.

Why the repair is safe

Operationally useful logging connects workflow state to data evidence. Preserve failed attempts, sanitize sensitive values, and design logs so another engineer can diagnose without reproducing your entire local setup.

Try it yourself

Take a simulated failed partition and write a six-field incident note: Dag/run, task, partition, attempt, observed error, next evidence to inspect. Then list three fields that should never be logged raw in a real system.

Fresh changed retry

A task is green but downstream total is wrong. Explain why the debugging path should move from orchestration status to data reconciliation/logic evidence rather than adding more retries.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Operationally useful logging connects workflow state to data evidence. Preserve failed attempts, sanitize sensitive values, and design logs so another engineer can diagnose without reproducing your entire local setup.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-18

Catchup, backfills and reprocessing

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Video assignment

Video assignment

Airflow 3 visual | Airflow 101 visual

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

Catchup and backfill are where sloppy date logic becomes expensive. Historical processing must target historical partitions, avoid duplicate side effects and preserve normal current state. Reprocessing should be an explicit operation, not 'change the computer clock and rerun.'

Picture it in your head

Imagine filling missing pages in a ledger. Each historical page has its own date label. You can fill pages 3 through 7 later, but you must write into those pages - not today's page five times.

Words before we use them

catchup = scheduler creates missed historical intervals from start boundary; backfill = intentionally run a historical range; reprocessing = rerun prior work; idempotent sink = repeated run converges to same logical state.

Follow with me once

Use the simulator with Sep 1-3, then delete only the Sep 2 output and rerun Sep 2. The partition total should return to the same value. Next run all three again and prove totals do not multiply. In Airflow, derive paths from data interval, not wall-clock time.

```
python practice/simulator/scripts/orchestration_simulator.py \
  --input-dir practice/simulator/input --output-dir practice/simulator/output
```

ic

Each date has one deterministic output file. Reprocessing Sep 2 replaces/converges that partition rather than appending a second logical copy. A historical range can be reasoned about before using Airflow's live backfill commands.

Break it / common failure

Do not backfill a non-idempotent external write without duplicate/transaction controls. Do not let historical runs overwrite a shared `latest.csv`. Do not assume `catchup=True` is always desirable; choose based on whether missed historical intervals should run.

Reason through a realistic incident

A backfill of seven days writes every run to `output/latest.json`. All seven runs succeed, but only the final file survives. The scheduler is green and the data history is destroyed. Partitioned output paths or idempotent upsert keys are therefore part of orchestration correctness. Historical run identity must survive into the side effect.

Why the repair is safe

Historical recovery is a first-class reliability feature. Partition identity, side-effect idempotency and explicit range controls determine whether backfill restores trust or creates duplicate damage.

Try it yourself

Design a backfill for Sep 1-7 where Sep 4 is known bad. State input/output partition naming, failure policy, whether later dates continue, and how you will reconcile after repair.

Fresh changed retry

A downstream API cannot safely accept the same event twice. Introduce an idempotency key or staging/upsert boundary and explain how that changes backfill safety.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

Historical recovery is a first-class reliability feature. Partition identity, side-effect idempotency and explicit range controls determine whether backfill restores trust or creates duplicate damage.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.

DE10-19

Integration and production-style orchestration

Watch first

[1. DataTalksClub / Timothy Davis - How to write data pipelines with Apache Airflow 3.0](#)

Optional visual - no verified bounded range

[2. Astronomer - Airflow 101: Essential Tips for Beginners \(retained from earlier route\)](#)

Optional visual - no verified bounded range; older visual, not current syntax authority

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Video assignment

Video assignment

Airflow 3 visual | Airflow 101 visual

[Airflow 3 visual - Timothy Davis - Open video](#) [Airflow 101 visual - Astronomer - Open video](#)

Optional visuals - no verified bounded range; Airflow 3.3.1 docs remain authority.

[Optional current-generation Airflow 3 module workshop exists in WATCH_GUIDE.md, but no bounded lesson segment is assigned because a reliable chapter boundary was not verified. Use the current official lesson](#) reference first.

Official/current reference [Open mapped source](#)

Open the lesson-matched current documentation. This is the primary source for the task.

Why this matters

The final M10 lesson combines container reproducibility with orchestration correctness. A production-style workflow is not 'Docker works' plus 'DAG is green'; it is a reproducible runtime whose historical runs, retries, configuration, side effects, evidence and recovery all agree with the data contract.

Picture it in your head

Picture a factory line inside a well-labeled portable building. Docker controls the repeatable building/runtime. Airflow controls when each station is allowed to work. Data-quality controls decide whether the product is acceptable. Logs and saved evidence let another operator explain and recover the line.

Words before we use them

reproducible environment = same declared build/topology from clean state; orchestration invariant = condition that must remain true across runs/retries; operational handoff = enough code/config/runbook/evidence for another engineer to operate safely.

Follow with me once

Complete two proofs. First, inspect the Docker project from a clean configuration: image recipe, service-name network boundary, output volume and no secret literals. Second, run the pure-Python orchestration simulator across Sep 1-3 including a one-time transient failure and a safe rerun. Then statically inspect the Airflow 3.3.1 Dag for `airflow.sdk`, interval-derived partitions, bounded retries, Params and small task communication.

```
python practice/run_static_qa.py
python practice/simulator/scripts/orchestration_simulator.py \
  --input-dir practice/simulator/input --output-dir practice/simulator/output
```

ic

Static/local QA proves the files are internally coherent and the simulator demonstrates deterministic partition behavior. It does NOT prove Docker Engine, Airflow scheduler/UI or DagRun behavior on the learner PC. Those remain explicit runtime checks before calling the module fully native-validated.

Break it / common failure

Do not use a passing static parser as evidence that Docker networking works. Do not use a green Airflow task as evidence that totals are correct. Do not hide environment blockers; record them and continue with the parts that can be truthfully validated.

Reason through a realistic incident

The final pipeline passes static checks and the simulator, but Docker Desktop has never been started and the Airflow Dag has never parsed on the learner PC. The correct status is not 'production ready'; it is 'static/local logic validated, native runtime pending'. After real runtime proof, keep the same discipline: Docker success, Airflow success and data reconciliation are three separate claims that all need evidence.

Why the repair is safe

M10 mastery means you can defend both layers: environment reproducibility and workflow reliability. The G04 gate tests fresh pipeline reasoning after this module; it is not passed by rereading the final lesson.

Try it yourself

Write a one-page operational handoff: clean setup, config/secrets ownership, Dag partition rule, retry/timeout policy, sensor/wait policy, data validation, backfill steps, evidence locations and rollback/recovery boundary. Another learner should be able to explain the design without your verbal help.

Fresh changed retry

The business switches from daily CSV to hourly object-store files. Identify which Docker pieces can remain unchanged, which Dag schedule/partition logic changes, and which validation/backfill controls must be retested.

Validation and recovery ladder

- State the intended partition/grain/side effect before changing code or configuration.
- Identify the earliest layer that disagrees with expectation: build/config/network/schedule/task/data/result.
- Inspect the smallest concrete evidence: resolved Compose config, container/task log, input partition, output file, count or total.
- Reveal only the next structural hint. Open the Explained Review only after a saved genuine attempt.
- Close review and complete the changed retry. Preserve the failed attempt beside the repair.

Ready-made revision

M10 mastery means you can defend both layers: environment reproducibility and workflow reliability. The G04 gate tests fresh pipeline reasoning after this module; it is not passed by rereading the final lesson.

Evidence / mastery

Save your actual command/config/code, observed output/error, one independent validation, a 60-90 second own-words explanation, and the changed retry. If live Docker/Airflow is required by the lesson, record it as PENDING until you actually run it on the learner PC - do not convert static evidence into a fake runtime PASS.